

ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ № 13

Թեմա՝ Պարզ Firewall համակարգի մոդելավորում

Առարկա: Ծրագրային անվտանգություն

Խումբ: SS-355 (Լաբ.-1)

Ուսանող: Մելիքյան Վերոնիկա

Outlook: veronikamelikyan.tt355@polytechnic.am

I. ՏԵՍԱԿԱՆ ՄԱՍ

1. Ինչ է Firewall-ը

Firewall-ը ծրագրային կամ ապարատային համակարգ է, որը վերահսկում է ցանցային տրաֆիկը՝ հիմնվելով անվտանգության կանոնների վրա:

Այն կատարում է հետևյալ գործողությունները՝

- մուտքային (incoming) տվյալների վերահսկում
- ելքային (outgoing) տվյալների վերահսկում
- պորտերի կառավարում (բացում / արգելափակում)
- վտանգավոր կապերի արգելափակում

2. Պորտերի գաղափարը

Պորտը (Port) ցանցային կապի տրամաբանական հասցե է, որը օգտագործվում է ծառայությունները միմյանցից տարբերելու համար: Հայտնի (well-known) պորտերի օրինակներ՝

Պորտ	Պրոտոկոլ
80	HTTP
443	HTTPS

Պորտ	Պրոտոկոլ
21	FTP
22	SSH
23	Telnet

Եթե պորտը անհայտ է, օգտագործվում է ընդհանուր *TCP/UDP* նշումը:

3. TCP և UDP

- TCP — կապի վրա հիմնված (connection-oriented), հուսալի փոխանցում, սխալների վերահսկում
- UDP — առանց կապի (connectionless), արագ, բայց ոչ հուսալի (չկա առաքման երաշխիք)

4. Firewall-ի հիմնական ֆունկցիաներ

- ✓ Պորտերի սկանավորում
- ✓ Պորտերի արգելափակում
- ✓ Պորտերի բացում
- ✓ Տրաֆիկի վերլուծություն

II. ԾՐԱԳՐԱՅԻՆ ՄԱՍ — PYTHON

Հիմնական հնարավորություններ

- ✓ Բաց պորտերի ցուցադրում
- ✓ Պորտերի արգելափակում (**receive / send**)
- ✓ Պորտերի բացում

✓ Ծանրաբեռնված պորտերի վերլուծություն

✓ Տվյալների պահպանում timestamp-ով JSON ֆայլում

Python ծրագրի ամբողջական կոդը

```
import psutil

import json

from datetime import datetime

from collections import Counter


PORT_PROTOCOLS = {

    80: "HTTP",

    443: "HTTPS",

    21: "FTP",

    22: "SSH",

    23: "Telnet",

}


firewall_rules = {

    "blocked_in": set(),

    "blocked_out": set(),

}


traffic_counter = Counter()
```

```

def scan_open_ports():

    """Վերադարձնում է բոլոր ակտիվ կապերի պորտերը:"""

    open_ports = set()

    for conn in psutil.net_connections(kind="inet"):

        if conn.laddr:

            port = conn.laddr.port

            open_ports.add(port)

            traffic_counter[port] += 1

    return open_ports


def get_protocol(port):

    return PORT_PROTOCOLS.get(port, "TCP/UDP")


def block_port(port, direction="in"):

    key = "blocked_in" if direction == "in" else "blocked_out"

    firewall_rules[key].add(port)

    print(f"[+] Port {port} blocked ({direction})")


def unblock_port(port, direction="in"):

    key = "blocked_in" if direction == "in" else "blocked_out"

    firewall_rules[key].discard(port)

    print(f"[-] Port {port} unblocked ({direction})")

```

```

def top_loaded_ports(n=5):

    return sorted(traffic_counter.items(),

                  key=lambda x: x[1], reverse=True)[:n]


def save_log():

    timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M")

    filename = f"firewall_log_{timestamp}.json"

    data = {

        "open_ports": list(scan_open_ports()),

        "top_loaded": top_loaded_ports(),

        "rules": {

            "blocked_in": list(firewall_rules["blocked_in"]),

            "blocked_out": list(firewall_rules["blocked_out"]),

        },

    }

    with open(filename, "w") as f:

        json.dump(data, f, indent=2)

    print(f"[*] Log saved → {filename}")


if __name__ == "__main__":

    ports = scan_open_ports()

```

```

print("Open ports:")

for p in sorted(ports):

    print(f"    {p:>5} → {get_protocol(p)}")


block_port(23, "in")          # Telnet receive արգելափակում

block_port(21, "out")         # FTP send արգելափակում


print("\nTop loaded ports:", top_loaded_ports())

save_log()

```

III. ԾՐԱԳՐԻ ԱՇԽԱՏԱՆՔԻ ՆԿԱՐԱԳՐՈՒԹՅՈՒՆ

Քայլ առ քայլ

I. Պորտերի սկանավորում

Ծրագիրը psutil գրադարանի միջոցով ստանում է համակարգի բոլոր ակտիվ ցանցային կապերը (psutil.net_connections):

2. Պրոտոկոլի որոշում

```
proto = PORT_PROTOCOLS.get(port, "TCP/UDP")
```

Եթե պորտը հայտնի է → վերադարձնում է HTTP, FTP, SSH և այլն. եթե ոչ → ընդհանուր TCP/UDP:

3. Firewall կանոններ

```

firewall_rules = {

    "blocked_in":  set(),

    "blocked_out": set(),

}

```

- blocked_in → մուտքային (receive) տվյալների արգելափակում

- `blocked_out` → ելքային (`send`) տվյալների արգելափակում

4. Տրաֆիկի հավաքագրում

Ծրագիրը `Counter`-ի միջոցով հաշվում է, թե յուրաքանչյուր պորտը քանի անգամ է օգտագործվել սկանավորման ընթացքում:

5. Ծանրաբեռնված պորտեր

```
sorted(traffic_counter.items(), key=lambda x: x[1], reverse=True)
```

Վերադարձնում է ամենաշատ օգտագործվող պորտերը՝ նվազման կարգով:

6. Տվյալների պահպանում

```
firewall_log_2026-05-04_14-30.json
```

JSON ֆայլը պարունակում է՝

- բաց պորտերի ցանկ
- ծանրաբեռնված պորտերի վիճակագրություն
- `firewall`-ի ակտիվ կանոններ

IV. LINUX — IPTABLES (ԻՐԱԿԱՆ FIREWALL)

Հիմնական գաղափար

C++-ից օգտագործում ենք `system()` կամ `exec()` կանչեր `iptables`-ի օգտագործմամբ՝ իրական `kernel`-մակարդակի `firewall` կանոններ ստեղծելու համար:

`iptables.cpp`

```
#include <iostream>
```

```
#include <cstdlib>
```

```
#include <string>
```

```
using namespace std;
```

```
void block_port(int port, const string& proto = "tcp") {
```

```

        string cmd = "sudo iptables -A INPUT -p " + proto +
                    " --dport " + to_string(port) + " -j DROP";

        cout << "[+] " << cmd << endl;

        system(cmd.c_str());
    }

    void unblock_port(int port, const string& proto = "tcp") {

        string cmd = "sudo iptables -D INPUT -p " + proto +
                    " --dport " + to_string(port) + " -j DROP";

        cout << "[-] " << cmd << endl;

        system(cmd.c_str());
    }

    void list_rules() {

        system("sudo iptables -L -n -v");
    }

    int main() {

        block_port(23);        // Telnet

        block_port(21);        // FTP

        list_rules();

        return 0;
    }

```

lin-firewall.cpp — բարձր մակարդակի logic

```

#include <iostream>

#include <fstream>

#include <set>

```



```

#include <ctime>

#include <cstdlib>


using namespace std;


set<int> blocked_in;
set<int> blocked_out;


void block(int port, const string& dir) {
    if (dir == "in")    blocked_in.insert(port);
    else                blocked_out.insert(port);
    cout << "Blocked " << port << " (" << dir << ")" << endl;
}


void save_log() {
    time_t t = time(nullptr);

    char buf[64];

    strftime(buf, sizeof(buf), "firewall_log_%Y-%m-%d_%H-%M.txt",
              localtime(&t));

    ofstream f(buf);

    f << "blocked_in: ";  for (int p : blocked_in)  f << p << " ";
    f << "\nblocked_out: "; for (int p : blocked_out) f << p << " ";
    cout << "Log → " << buf << endl;
}


int main() {
    block(23, "in");

    block(21, "out");
}

```

```
    save_log();  
    return 0;  
}
```

V. ԻՐԱԿԱՆ ԱՐԳԵԼԱՓԱԿՈՒՄ — ՀՐԱՄԱՆԱՅԻՆ ՏՈՂ

Linux (iptables)

```
sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

Windows (PowerShell)

```
New-NetFirewallRule -DisplayName "Block 80" `
    -Direction Inbound -LocalPort 80 `
    -Protocol TCP -Action Block
```

VI. ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔԻ ՆՊԱՏԱԿ

Ուսանողը պետք է կարողանա՝

- ✓ հասկանալ պորտերի աշխատանքը
- ✓ տարբերել TCP և UDP պրոտոկոլները
- ✓ ստեղծել պարզ firewall logic
- ✓ վերլուծել ցանցային տրաֆիկը

VII. ԼՐԱՑՈՒՑԻՉ ԱՌԱՋԱԴՐԱՆՔՆԵՐ

Աշխատանքը կարելի է ընդլայնել հետևյալ ուղղություններով՝

1. GUI ավելացում (Tkinter Python-ի համար, Qt՝ C++-ի)
2. Իրական packet filtering scapy / libpcap գրադարաններով
3. Linux iptables-ի ինտեգրում որպես իրական backend
4. Logging + visualization (matplotlib, Grafana)
5. Real-time monitoring thread
6. IDS behavior — port scan-ի ավտոմատ հայտնաբերում

VIII. ԱՐԴՅՈՒՆՔ

- ✓ OS-կիրառական մակարդակում firewall-ի կիրառում
- ✓ Հասկացել ենք firewall-ի աշխատանքի սկզբունքները նաև ծրագրային մակարդակում
- ✓ Աշխատել ենք թե՛ Windows, թե՛ Linux միջավայրերում
- ✓ Իրականացրել ենք իրական համակարգային ծրագրավորման օրինակ